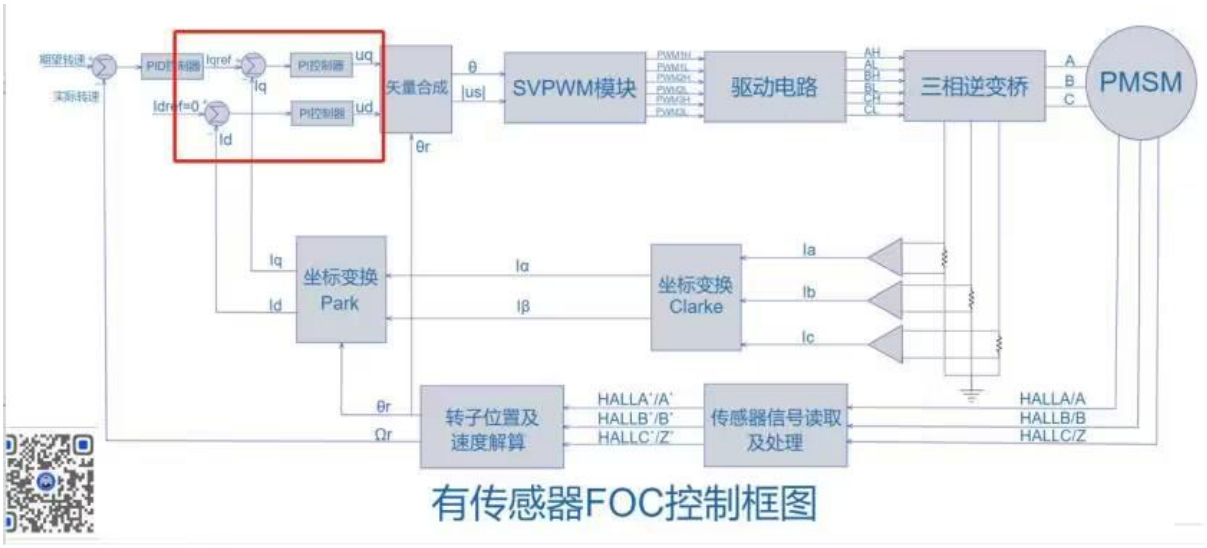
61-FOC 14

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。



上节课，我们讲了电流环，电流控制环执行后，得到 Ud 跟 Uq。这一步，有的学员，觉得不好理解，实际上 PID 控制可用于线性非时变系统，这里的线性也可以是近似线性，系统具备因果性和稳定性，系统输出只与当前以及过去的输入有关，系统对每个输入和输出来说都是有界的，系统是有界输入有界输出稳定的，这样的系统就可以用 PID 控制。前人根据经验的发明和总结，其实我们也许只了解了冰山一角，控制理论深入下去很复杂，我们课程仅是简单的应用。

后面呢，就需要实现矢量合成模块。矢量合成模块的话，它对应两个输入，以及两个输出。其中|Us|这个输出值的话，就利用我们之前讲过的求平方根实现。而 θ 的话，会用到公式 θ = θr + 90° + θc 或者 θ = θr - 90° - θc
这个 θc，对应就是超前角。超前角的求解呢，就会用到我们前几次课讲的 Cordic 数字迭代算法。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
File          : Voltage.c
Function      : 电压计算
Author        : 张飞实战电子
*/
#include "..\Include\Voltage.h"
/*
void GetUs(void)
{
    unsigned int Temp;
    signed int qVq_Max;

    tUm.qUsMax = (signed int)(tUm.PercentMax >> 1);
    Temp = (unsigned int)(MulFrac(tUm.qUsMax, tUm.qUsMax) - MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    Temp = CalSqrt(Temp);
    qVq_Max = (signed int)(Temp >> 1);

    if (tUm.qUq > qVq_Max)
        tUm.qUq = qVq_Max;
    if (tUm.qUq < -qVq_Max)
        tUm.qUq = -qVq_Max;

    Temp = (unsigned int)(MulFrac(tUm.qUq, tUm.qUq) + MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    tUm.PercentOut = CalSqrt(Temp);

    if(tFoc.Direction == CCW)
        tUm.qDQAngle = CalAtan(-tUm.qUd, tUm.qUq);
    else
        tUm.qDQAngle = CalAtan(-tUm.qUd, -tUm.qUq);
}
/*==========版权归张飞实战电子所有，违者必究！！！==========*/
```

所以，这里大家来看下这个函数，它的功能就是用于求取电压矢量模长跟超前角。这个函数中，调用了求平方根　跟　计算反正切的函数。这两个函数呢，我们需要先把之前讲的理论转换为对应的代码。这里呢，我们也可以先整体了解一下这个函数，然后再分析内部的子函数。

```
/*
void GetUs(void)
{
    unsigned int Temp;
    signed int qVq_Max;

    tUm.qUsMax = (signed int)(tUm.PercentMax >> 1);
    Temp = (unsigned int)(MulFrac(tUm.qUsMax, tUm.qUsMax) - MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    Temp = CalSqrt(Temp);
    qVq_Max = (signed int)(Temp >> 1);

    if (tUm.qUq > qVq_Max)
        tUm.qUq = qVq_Max;
    if (tUm.qUq < -qVq_Max)
        tUm.qUq = -qVq_Max;

    Temp = (unsigned int)(MulFrac(tUm.qUq, tUm.qUq) + MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    tUm.PercentOut = CalSqrt(Temp);
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

而这里求解|Us|的思路，是需要先对 Uq 进行限幅，然后用限幅的 Uq，再结合 Ud 来求解 Us。

```
void GetUs(void)
{
    unsigned int Temp;
    signed int qVq_Max;

    tUm.qUsMax = (signed int)(tUm.PercentMax >> 1);
    Temp = (unsigned int)(MulFrac(tUm.qUsMax, tUm.qUsMax) - MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    Temp = CalSqrt(Temp);
    qVq_Max = (signed int)(Temp >> 1);

    if (tUm.qUq > qVq_Max)
        tUm.qUq = qVq_Max;
    if (tUm.qUq < -qVq_Max)
        tUm.qUq = -qVq_Max;

    Temp = (unsigned int)(MulFrac(tUm.qUq, tUm.qUq) + MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    tUm.PercentOut = CalSqrt(Temp);
```

所以，单纯看红框这个部分，就是求 Uq 跟 Ud 的平方，然后加起来。加完之后的值，要调用求平方根函数，得到对应的|Us|。

```
/*=============================================================================*/
void GetUs(void)
{
    unsigned int Temp;
    signed int qVq_Max;

    tUm.qUsMax = (signed int)(tUm.PercentMax >> 1);
    Temp = (unsigned int)(MulFrac(tUm.qUsMax, tUm.qUsMax) - MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    Temp = CalSqrt(Temp);
    qVq_Max = (signed int)(Temp >> 1);

    if (tUm.qUq > qVq_Max)
        tUm.qUq = qVq_Max;
    if (tUm.qUq < -qVq_Max)
        tUm.qUq = -qVq_Max;

    Temp = (unsigned int)(MulFrac(tUm.qUq, tUm.qUq) + MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    tUm.PercentOut = CalSqrt(Temp);
```

我们对 Uq 进行了限幅处理，这是因为，电压矢量的模长实际上是达不到 1 的。而考虑到 PWM 互补调制，还要添加上死区时间，所以呢，这个矢量的模长，一般是 92%左右。而我们的控制方式是 Id = 0 控制，所以，电流矢量 Is 是落在 q 轴上了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

那么，电压矢量跟电流矢量虽然存在夹角，但电压矢量是更靠近电流矢量的，所以呢，我们限幅的思路可以是限制 Uq 的大小。因为，Us 在 dq 轴的投影，Us 更加靠近 q 轴的话，那么 Uq 就越接近最大值，Ud 的话，相对是比较小的。所以啊，我们把 Uq 限制住最大值，自然对应的 Us 也被限制住了。

```
/*-------------------------------------------------------------------------*/
void GetUs(void)
{
    unsigned int Temp;
    signed int qVq_Max;

    tUm.qUsMax = (signed int)(tUm.PercentMax >> 1);
    Temp = (unsigned int)(MulFrac(tUm.qUsMax, tUm.qUsMax) - MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    Temp = CalSqrt(Temp);
    qVq_Max = (signed int)(Temp >> 1);

    if (tUm.qUq > qVq_Max)
        tUm.qUq = qVq_Max;
    if (tUm.qUq < -qVq_Max)
        tUm.qUq = -qVq_Max;

    Temp = (unsigned int)(MulFrac(tUm.qUq, tUm.qUq) + MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    tUm.PercentOut = CalSqrt(Temp);
```

大家来看下这句话，tUm.PercentMax 这个值我们之前讲 SVPWM 的时候，设定了它对应的是 92%的值。对于无符号的 0.16 格式的数据，0~0.9999 对应 0~65535，92%的话，也就对应 60293。所以，限幅的最大值就使用它。但是这个地方因为 Ud　Uq 都是有符号的 Q15 格式的数据。

所以呢，为了数据格式的统一，我们可以把这个无符号的最大值转换为有符号的最大值。而对于有符号的 Q15，-1~0.999 对应　-32768~32767。所以，从两个格式表达的 0~0.999 来看，无符号对应的是 0~65535，有符号对应的是　0~32767。所以，我们要把无符号的 0~0.999 转换为有符号的 0~0.999 的话，实际上就是把无符号的数据除 2 来得到。除 2 的话，可以使用右移 1 位来代替。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
void GetUs(void)
{
    unsigned int Temp;
    signed int qVq_Max;

    tUm.qUsMax = (signed int)(tUm.PercentMax >> 1);
    Temp = (unsigned int)(MulFrac(tUm.qUsMax, tUm.qUsMax) - MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    Temp = CalSqrt(Temp);
    qVq_Max = (signed int)(Temp >> 1);

    if (tUm.qUq > qVq_Max)
        tUm.qUq = qVq_Max;
    if (tUm.qUq < -qVq_Max)
        tUm.qUq = -qVq_Max;

    Temp = (unsigned int)(MulFrac(tUm.qUq, tUm.qUq) + MulFrac(tUm.qUd, tUm.qUd));
```

所以呢，这句话的作用，就实现 0.16~0.15 格式的转换。这样的话，就把矢量的最大值转换为了有符号来表示。也就是使用 tUm.qUsMax 来表示矢量的最大值了。

```
void GetUs(void)
{
    unsigned int Temp;
    signed int qVq_Max;

    tUm.qUsMax = (signed int)(tUm.PercentMax >> 1);
    Temp = (unsigned int)(MulFrac(tUm.qUsMax, tUm.qUsMax) - MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    Temp = CalSqrt(Temp);
    qVq_Max = (signed int)(Temp >> 1);

    if (tUm.qUq > qVq_Max)
        tUm.qUq = qVq_Max;
    if (tUm.qUq < -qVq_Max)
        tUm.qUq = -qVq_Max;
```

接着呢，我们使用最大值的平方，减去 Ud 的平方，可以得到 Uq 的平方。这个平方值的话，就全是正值了。所以，可以使用一个无符号的变量来存储。而之前我们讲解的求平方根的思路，输入的数据是 x 对应的是 0~1 也就是无符号的数据。对应的范围呢，也就是 0~65535。所以啊，我们需要把 Uq 的平方值，转换为 0.16 格式的数据。这个转换呢，也就对应 Temp <<= 1 这句话了。也就是说，把 Q15 格式的数据乘 2，或者左移 1 位就可以。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
void GetUs(void)
{
    unsigned int Temp;
    signed int qVq_Max;

    tUm.qUsMax = (signed int)(tUm.PercentMax >> 1);
    Temp = (unsigned int)(MulFrac(tUm.qUsMax, tUm.qUsMax) - MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    Temp = CalSqrt(Temp);
    qVq_Max = (signed int)(Temp >> 1);
```

然后呢，对 Uq 的平方值开一个平方根，就可以得到 Uq。而实际的 Uq 呢，是一个有符号的量，所以为了方便比较，需要把开平方得到的 Uq 转换为有符号的。因此，又进行了一次转换，把无符号的 Uq 转换为有符号的 Uq。

```
void GetUs(void)
{
    unsigned int Temp;
    signed int qVq_Max;

    tUm.qUsMax = (signed int)(tUm.PercentMax >> 1);
    Temp = (unsigned int)(MulFrac(tUm.qUsMax, tUm.qUsMax) - MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    Temp = CalSqrt(Temp);
    qVq_Max = (signed int)(Temp >> 1);
```

这样的话，也就得到 Uq 的限幅值。这里，大家需要重点关注的是数据格式的转换。大家看下，根据 Us 最大值求解 Uq 最大值的这个代码实现，可以理解吧？ 当然了，关于开平方根的函数，我们一会再来分析啊，先把这个函数当成已知的，这里直接调用。

大家看下，根据 Us 最大值求解 Uq 最大值的这个代码实现，可以理解吧？

问：为什么又从 Q15 转换成 Q16 呢？

答：求平方根的函数是 Q16 输入的。

```
void GetUs(void)
{
    unsigned int Temp;
    signed int qVq_Max;

    tUm.qUsMax = (signed int)(tUm.PercentMax >> 1);
    Temp = (unsigned int)(MulFrac(tUm.qUsMax, tUm.qUsMax) - MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    Temp = CalSqrt(Temp);
    qVq_Max = (signed int)(Temp >> 1);

    if (tUm.qUq > qVq_Max)
        tUm.qUq = qVq_Max;
    if (tUm.qUq < -qVq_Max)
        tUm.qUq = -qVq_Max;
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

大家看，Uq 的最大值求解出来之后，用电流环的输出值跟最大值比较，就可以得到限幅之后的 Uq。我们呢，就可以使用这个 Uq 跟 Ud 求解 Us 的模长了。

问：这时求取出来的 qVq_Max 是 Uq 最大值的绝对值？
答：是的。

```
tUm.qUsMax = (signed int)(tUm.PercentMax >> 1);
Temp = (unsigned int)(MulFrac(tUm.qUsMax, tUm.qUsMax) - MulFrac(tUm.qUd, tUm.qUd));
Temp <<= 1;
Temp = CalSqrt(Temp);
qVq_Max = (signed int)(Temp >> 1);

if (tUm.qUq > qVq_Max)
    tUm.qUq = qVq_Max;
if (tUm.qUq < -qVq_Max)
    tUm.qUq = -qVq_Max;

Temp = (unsigned int)(MulFrac(tUm.qUq, tUm.qUq) + MulFrac(tUm.qUd, tUm.qUd));
Temp <<= 1;
tUm.PercentOut = CalSqrt(Temp);
```

求解的方法呢，就是 sqrt(Uq^2 + Ud^2)

$$|Us| = \sqrt{U_d^2 + U_q^2}$$

这里，又涉及到数据格式的转换。先把有符号的 Us^2 求解出来，然后转换为 0.16 格式，然后调用开平方根的函数，就可以得到 Us 的模长。这个模长呢，是 0.16 格式的。

```
if(tFoc.Direction == CCW)
    tUm.qDQAngle = CalAtan(-tUm.qUd, tUm.qUq);
else
    tUm.qDQAngle = CalAtan(-tUm.qUd, -tUm.qUq);
```

然后呢，得到了 Ud Uq，我们就可以利用反正切把超前角给求解出来。对应的公式呢，我们之前给大家列过，这里反正切的实现，要用到我们前面讲的 Cordic 数字迭代算法。反正切对应的函数呢，我们分析完开平方根的函数，再来分析哈。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
void GetUs(void)
{
    unsigned int Temp;
    signed int qVq_Max;

    tUm.qUsMax = (signed int)(tUm.PercentMax >> 1);
    Temp = (unsigned int)(MulFrac(tUm.qUsMax, tUm.qUsMax) - MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    Temp = CalSqrt(Temp);
    qVq_Max = (signed int)(Temp >> 1);

    if (tUm.qUq > qVq_Max)
        tUm.qUq = qVq_Max;
    if (tUm.qUq < -qVq_Max)
        tUm.qUq = -qVq_Max;

    Temp = (unsigned int)(MulFrac(tUm.qUq, tUm.qUq) + MulFrac(tUm.qUd, tUm.qUd));
    Temp <<= 1;
    tUm.PercentOut = CalSqrt(Temp);

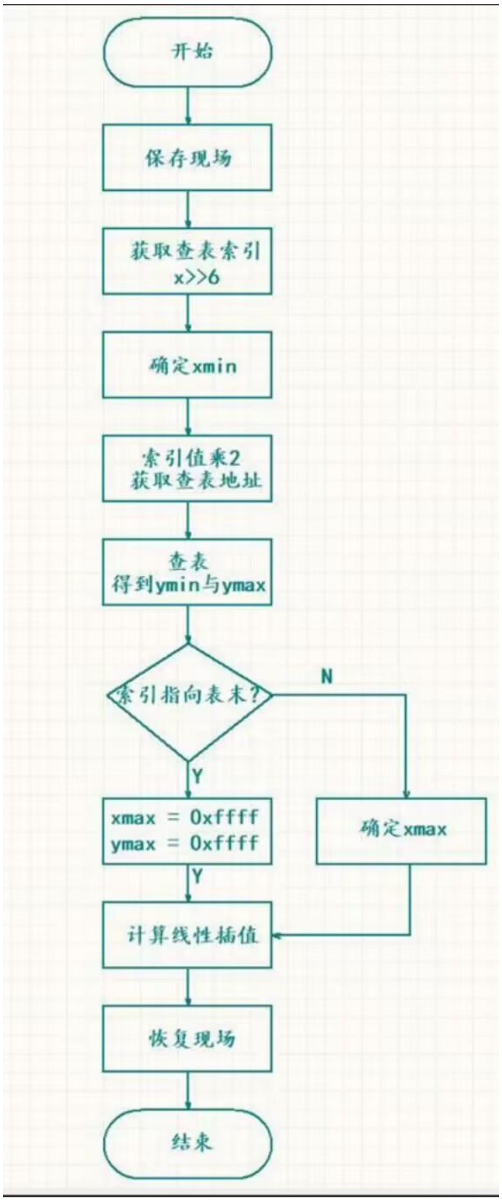
    if(tFoc.Direction == CCW)
        tUm.qDQAngle = CalAtan(-tUm.qUd, tUm.qUq);
    else
        tUm.qDQAngle = CalAtan(-tUm.qUd, -tUm.qUq);
}
```

所以啊，这就是我们求解电压矢量模长　以及电压矢量跟电流矢量之间的夹角的代码。好的，那我们下面来分析下开平方根的代码啊。

```
/*=============================================================
extern signed int MulFrac(signed int x,signed int y);
extern unsigned int CalSqrt(unsigned int x);
extern signed int CalAtan(signed int y,signed int x);
/*=============================================================
```

先来看下这个函数的调用声明。它是含有一个形参及一个返回值的，两个值呢，都是无符号的。我们就是要求解 x（0~1）的平方根　y(0~1)。这个函数呢，我们也是用汇编来编写的，主要的思路是查表+线性差值。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



这是当时绘制出来的流程图，编写程序呢，就根据这个流程图来的。

```
8   Author          : 张飞实战电子
9   ================================================================================*/
10      .include    "../Include/p33EP32MC204.inc"
11
12      .global     _MulFrac
13      .global     _CalSqrt
14      .global     _CalAtan
15  /*==============================================================================*/
16  ;符号定义
```

因为这个汇编函数需要在 C 文件中进行调用，所以需要在头文件中进行全局声明。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*=============================================================================*/
_CalSqrt:
    push        DSRPAG

    mov         W0,W1
    lsr         W1,#6,W6   ;获取索引号
    mov         #0xffc0,W0
    and         W1,W0,W2   ;获取xmin

    add         W6,W6,W7   ;根据索引号获取查表地址

    movpag      #psvpage(_Sqrt_Table),DSRPAG
    mov         #psvoffset(_Sqrt_Table),W0    ;获取页内表数据存放首地址
```

现在我们来分析对应的代码啊。因为查表需要用到 DSRPAG 寄存器，所以呢，我们这里先把这个寄存器压栈保存。这样的话，不会破坏它原始的值。然后，w0 这个寄存器中，存储的就是形参 x 对应的值。首先呢，我们要获取查表的索引号，求索引号的方法呢，是使用 x 右移 6 位。

```
_CalSqrt:
    push        DSRPAG

    mov         W0,W1
    lsr         W1,#6,W6   ;获取索引号
```

所以，大家看，这里就使用了一条逻辑移位指令 lsr，对 w1 右移 6 位，把移位后的结果存放在 w6 中。也就是说，w6 寄存器中存储了索引号。接着呢，我们要求解 xmin，获取 xmin 的话，是用 x 与上 0xffc0。

```
/*=============================================================================*/
_CalSqrt:
    push        DSRPAG

    mov         W0,W1
    lsr         W1,#6,W6   ;获取索引号
    mov         #0xffc0,W0
    and         W1,W0,W2   ;获取xmin
```

所以，这里用两条汇编指令，就得到 xmin，结果呢，是存在 w2 寄存器中的。接着呢，我们要根据索引号去查表，查表得到 ymin 跟 ymax。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
_CalSqrt:
    push        DSRPAG

    mov         W0,W1
    lsr         W1,#6,W6   ;获取索引号
    mov         #0xffc0,W0
    and         W1,W0,W2   ;获取xmin

    add         W6,W6,W7   ;根据索引号获取查表地址

    movpag      #psvpage(_Sqrt_Table),DSRPAG
    mov         #psvoffset(_Sqrt_Table),W0    ;获取页内表数据存放首地址

    add         W7,W0,W0
    mov         [W0++],W4        ;ymin
    mov         [W0],W5          ;ymax
```

查表的话，就根据索引号来查，但是呢，这里需要注意一下，数据在 Flash 中存储是偶数对齐的。所以，我们要查表的话，需要根据索引号转换为查表地址。方法呢，就是把索引号*2，只不过，这里的*2 是用加法实现的。

```
mov         W0,W1
lsr         W1,#6,W6   ;获取索引号
mov         #0xffc0,W0
and         W1,W0,W2   ;获取xmin

add         W6,W6,W7   ;根据索引号获取查表地址

movpag      #psvpage(_Sqrt_Table),DSRPAG
mov         #psvoffset(_Sqrt_Table),W0    ;获取页内表数据存放首地址

add         W7,W0,W0
mov         [W0++],W4        ;ymin
mov         [W0],W5          ;ymax
```

因为 w6 存放的是索引号，w6+w6 就相当于索引号*2，也就是说，w7 存放的是查表地址。

```
mov         W0,W1
lsr         W1,#6,W6   ;获取索引号
mov         #0xffc0,W0
and         W1,W0,W2   ;获取xmin

add         W6,W6,W7   ;根据索引号获取查表地址

movpag      #psvpage(_Sqrt_Table),DSRPAG
mov         #psvoffset(_Sqrt_Table),W0    ;获取页内表数据存放首地址

add         W7,W0,W0
mov         [W0++],W4        ;ymin
mov         [W0],W5          ;ymax

inc         W6,W6    ;索引号加1
```

红框中，就是查表得到 ymin 跟 ymax。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
_CalSqrt:
    push        DSRPAG

    mov         W0,W1
    lsr         W1,#6,W6   ;获取索引号
    mov         #0xffc0,W0
    and         W1,W0,W2   ;获取xmin

    add         W6,W6,W7   ;根据索引号获取查表地址

    movpag      #psvpage(_Sqrt_Table),DSRPAG
    mov         #psvoffset(_Sqrt_Table),W0    ;获取页内表数据存放首地址

    add         W7,W0,W0
    mov         [W0++],W4       ;ymin
    mov         [W0],W5         ;ymax
```

那么，分析到这里，我们根据 x 获取得到了索引号、xmin，查表得到了 ymin ymax。

```
    movpag      #psvpage(_Sqrt_Table),DSRPAG
    mov         #psvoffset(_Sqrt_Table),W0    ;获取页内表数据存放首地址

    add         W7,W0,W0
    mov         [W0++],W4       ;ymin
    mov         [W0],W5         ;ymax
```

这个查表好像很神奇，操作 FLASH 中的数据就好像在操作 RAM 中的数据一样，这是这个单片机具有特殊的影射机制，就是使用 DSRPAG 进行 PSV 操作。

关于 PSV 操作，视频中也有详细讲解的。顺便说一下，这种 PSV 操作，一般单片机是不具备这个功能的。

问：咱们对于这个具体实现是不需要知道的吧，只要知道片子有这个功能，按照他们说明这么去操作就可以了是这样么？

答：是的。

问：W4,W6,W7,这些寄存器，在调用这个汇编函数之前，可能被用了 C 语言的代码用了啊，在这个汇编函数开始的时候，W6,W7 可以不压栈，就用吗？

答：C 语言只使用 W0~W3 寄存器，这 4 个寄存器属于影子寄存器，在调用中断的时候，硬件自动压栈了。

问：那也只是 W0~W3,会自动压栈,W4,w5 也会自动压栈？

答：C 语言是编译器考虑压栈问题的，如果是汇编，全程需要用户考虑压栈细节。C 语言不需要用户考虑压栈问题，换句话说，就是压栈问题编译器如何处理，对用户而言就是黑盒子。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

好，我们继续往下讲了。

```
    inc         W6,W6   ;索引号加1
    mov         #0x03ff,W0
    and         W6,W0,W6 ;跟1023相与
    bra         NZ,_CalSqrt_Norm
    mov         #0xffff,W3 ;xmax    索引已经指向表末尾
    mov         #0xffff,W5  ;ymax
    goto        _CalSqrt_go
_CalSqrt_Norm:
    mov         #64,W0
    add         W0,W2,W3    ;xmax
_CalSqrt_go:
```

接下来，要处理一种特殊情况。之前我们分析过，索引可能会指向表的末尾。这样的话，我们就不能得到 ymax，查表得到的 ymax 不对。所以，碰到这种情况我们需要特殊处理，我们需要判断一下，是不是出现了这种情况。

```
    inc         W6,W6   ;索引号加1
    mov         #0x03ff,W0
    and         W6,W0,W6 ;跟1023相与
    bra         NZ,_CalSqrt_Norm
    mov         #0xffff,W3 ;xmax    索引已经指向表末尾
    mov         #0xffff,W5  ;ymax
    goto        _CalSqrt_go
_CalSqrt_Norm:
    mov         #64,W0
    add         W0,W2,W3    ;xmax
_CalSqrt_go:
```

这里呢，我们先把索引号加 1，然后用这个值跟 0x3ff 相与。如果说与的结果不是 0，则说明索引没有指向表的末尾，如果说，与的结果是 0，则说明索引指向了表的末尾。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    mov         [W0],W5          ;ymax

    inc         W6,W6   ;索引号加1
    mov         #0x03ff,W0
    and         W6,W0,W6 ;跟1023相与
    bra         NZ,_CalSqrt_Norm
    mov         #0xffff,W3 ;xmax    索引已经指向表末尾
    mov         #0xffff,W5  ;ymax
    goto        _CalSqrt_go
_CalSqrt_Norm:
    mov         #64,W0
    add         W0,W2,W3    ;xmax
_CalSqrt_go:
```

大家看，如果说与的结果不是 0，就会跳转到红色的箭头。此时呢，xmax 就是 xmin + 64， xmax 是存储在 w3 寄存器中的。

```
    inc         W6,W6   ;索引号加1
    mov         #0x03ff,W0
    and         W6,W0,W6 ;跟1023相与
    bra         NZ,_CalSqrt_Norm
    mov         #0xffff,W3 ;xmax    索引已经指向表末尾
    mov         #0xffff,W5  ;ymax
    goto        _CalSqrt_go
_CalSqrt_Norm:
    mov         #64,W0
    add         W0,W2,W3    ;xmax
_CalSqrt_go:
```

如果说索引指向了表末，则 xmax ymax 就会直接赋值 65535。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    movpag      #psvpage(_Sqrt_Table),DSRPAG
    mov         #psvoffset(_Sqrt_Table),W0    ;获取页内表数据存放首地址

    add         W7,W0,W0
    mov         [W0++],W4       ;ymin
    mov         [W0],W5         ;ymax

    inc         W6,W6  ;索引号加1
    mov         #0x03ff,W0
    and         W6,W0,W6 ;跟1023相与
    bra         NZ,_CalSqrt_Norm
    mov         #0xffff,W3 ;xmax   索引已经指向表末尾
    mov         #0xffff,W5  ;ymax
    goto        _CalSqrt_go
_CalSqrt_Norm:
    mov         #64,W0
    add         W0,W2,W3    ;xmax
_CalSqrt_go:

    sub         W1,W2,W6 ;x - xmin
    sub         W3,W2,W3 ;xmax - xmin
    sub         W5,W4,W5 ;ymax -ymin

    mul.uu      W6,W5,W6
    repeat      #17
    div.ud      W6,W3    ;w0 -> y - ymin
```

那么，处理完之后呢，就会跳转到_CalSqrt_go 这个标号。到目前为止呢，x xmin ymin ymax 的值都求解出来了。

$$\frac{x - x_{min}}{x_{max} - x_{min}} = \frac{y - y_{min}}{y_{max} - y_{min}}$$

$$y - y_{min} = \frac{x - x_{min}}{x_{max} - x_{min}} (y_{max} - y_{min})$$

然后，我们根据这个公式，可以把 y - ymin 给求解出来。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
add         W0,W2,W3    ;xmax
_CalSqrt_go:

sub         W1,W2,W6 ;x - xmin
sub         W3,W2,W3 ;xmax - xmin
sub         W5,W4,W5 ;ymax -ymin

mul.uu      W6,W5,W6
repeat      #17
div.ud      W6,W3    ;w0 -> y - ymin

add         W4,W0,W0        ;y = y - ymin + ymin

pop         DSRPAG

return
```

大家看，这里就先计算了三个减法，然后呢，接着就把 （x - xmin）* （ymax - ymin）求解出来。最后呢，再求解除法。

```
add         W0,W2,W3    ;xmax
_CalSqrt_go:

sub         W1,W2,W6 ;x - xmin
sub         W3,W2,W3 ;xmax - xmin
sub         W5,W4,W5 ;ymax -ymin

mul.uu      W6,W5,W6
repeat      #17
div.ud      W6,W3    ;w0 -> y - ymin

add         W4,W0,W0        ;y = y - ymin + ymin

pop         DSRPAG

return
```

这里乘法的话，使用的是一条乘法指令。乘法的结果呢，是存放在 w6 w7 中的，因为两个 16 位的数相乘，得到的结果需要 32 位来存放，所以需要两个 w 寄存器来存储。

MUL.UU	Wb,Ws,Wnd	{Wnd + 1, Wnd} = 无符号 (Wb) * 无符号 (Ws)	1	1(5)	320

这是这个乘法指令的原型。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
_CalSqrt_go:

    sub         W1,W2,W6 ;x - xmin
    sub         W3,W2,W3 ;xmax - xmin
    sub         W5,W4,W5 ;ymax -ymin

    mul.uu      W6,W5,W6
    repeat      #17
    div.ud      W6,W3    ;w0 -> y - ymin

    add         W4,W0,W0         ;y = y - ymin + ymin

    pop         DSRPAG

    return
```

然后，接下来要计算一个除法。除法指令的话，执行周期比较长，所以，除法指令必须联合 repeat 指令来使用，也就是要迭代 18 次，才能得到最终的结果。

DIV.UD	Wm, Wn	无符号 32/16 位整数除法， Q → W0， R → W1	1	18(2)	226

```
tUm.qUsMax = (signed int)(tUm.PercentMax >> 1);
Temp = (unsigned int)(MulFrac(tUm.qUsMax,tUm.qUsMax) - MulFrac(tUm.qUd,tUm.qUd));
Temp <<= 1;
Temp = CalSqrt(Temp);
qVq_Max = (signed int)(Temp >> 1);

if (tUm.qUq > qVq_Max)
    tUm.qUq = qVq_Max;
if (tUm.qUq < -qVq_Max)
    tUm.qUq = -qVq_Max;
```

问：这个得到的是 uq 的平方，就是 Q15*Q15=Q30，左移一位变成 Q31？怎么是 Q15 左移一位变成 Q16 呢？

答：开方函数输入参数是 Q16 格式的。

```
if(tFoc.Direction == CCW)
    tUm.qDQAngle = CalAtan(-tUm.qUd,tUm.qUq);
else
    tUm.qDQAngle = CalAtan(-tUm.qUd,-tUm.qUq);
```

问：1 是逆时针第二扇区，2 公式是什么原因呢？

答：1 公式是 CCW， 2 公式是 CW。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这个是除法指令的原型。用于计算 32 位除以 16 位。除法的商是存在 w0 中的，余数是存放在 w1 寄存器中。我们这里呢，只需要商，所以取 w0 的结果。

语法：{标号:} DIV.U{W} Wm, Wn
DIV.UD Wm, Wn

操作数：对于字操作， Wm ∈ [W0 ... W15]
对于双字操作， Wm ∈ [W0, W2, W4 ... W14]
Wn ∈ [W2 ... W15]

操作：对于字操作（默认）：
Wm → W0
0x0 → W1
W1:W0/Wn → W0
余数 → W1

对于双字操作（DIV.UD）：
Wm + 1:Wm → W1:W0
W1:W0/Wns → W0
余数 → W1

受影响的状态位：N、OV、Z 和 C

指令编码：

1101	1000	lttt	tvvv	vW00	ssss

描述：迭代的无符号整数除法，其中被除数存储在 Wm（对于 16 位 /16 位除法）或 Wm + 1:Wm（对于 32 位 /16 位除法）中，而除数存储在 Wn 中。在字操作中，Wm 将首先被拷贝至 W0，而 W1 将被清零以执行除法操作。在双字操作中，Wm + 1:Wm 将首先被拷贝至 W1:W0。除法操作的 16 位商将存放在 W0 中，而 16 位余数则被存放在 W1 中。
必须通过 REPEAT 指令（迭代计数值为 17）执行该指令 18 次以得到正确的商和余数。N 标志总是被清零。如果除法操作导致溢出，则 OV 标志将被置 1，否则将被清零。如果余数为 0，则 Z 标志将被置 1，否则将被清零。
C 标志用来实现除法算法，但不应使用其最终值。
t 位用于选择双字操作中被除数的高位字。对于字操作，这些位将被清零。

大家可以了解一下，这是除法指令的语法描述。

```
_CalSqrt_go:

    sub         W1,W2,W6 ;x - xmin
    sub         W3,W2,W3 ;xmax - xmin
    sub         W5,W4,W5 ;ymax -ymin

    mul.uu      W6,W5,W6
    repeat      #17
    div.ud      W6,W3    ;w0 -> y - ymin

    add         W4,W0,W0         ;y = y - ymin + ymin

    pop         DSRPAG

    return
```

w0 中存储的是 y-ymin 的值，而 ymin 的值之前查表得出是存储在 w4 寄存器中的。所以，w4 加 w0，就可以得到 y 的值。这个结果呢，是存在 w0 中的，这个 w0 的话，也就是函数的返回值。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    mul.uu      W6,W5,W6
    repeat      #17
    div.ud      W6,W3    ;w0 -> y - ymin

    add         W4,W0,W0         ;y = y - ymin + ymin

    pop         DSRPAG

    return
```

最后呢，把 DSRPAG 存储的值弹出堆栈，恢复原始值。最终，加上子函数返回指令，整个函数就结束了。

```
/*=====================================================================*/
_CalSqrt:
    push        DSRPAG

    mov         W0,W1
    lsr         W1,#6,W6  ;获取索引号
    mov         #0xffc0,W0
    and         W1,W0,W2  ;获取xmin

    add         W6,W6,W7  ;根据索引号获取查表地址

    movpag      #psvpage(_Sqrt_Table),DSRPAG
    mov         #psvoffset(_Sqrt_Table),W0   ;获取页内表数据存放首地址

    add         W7,W0,W0
    mov         [W0++],W4       ;ymin
    mov         [W0],W5         ;ymax

    inc         W6,W6  ;索引号加1
    mov         #0x03ff,W0
    and         W6,W0,W6 ;跟1023相与
    bra         NZ,_CalSqrt_Norm
    mov         #0xffff,W3 ;xmax   索引已经指向表末尾
    mov         #0xffff,W5  ;ymax
    goto        _CalSqrt_go
_CalSqrt_Norm:
    mov         #64,W0
    add         W0,W2,W3   ;xmax
_CalSqrt_go:

    sub         W1,W2,W6 ;x - xmin
    sub         W3,W2,W3 ;xmax - xmin
    sub         W5,W4,W5 ;ymax -ymin

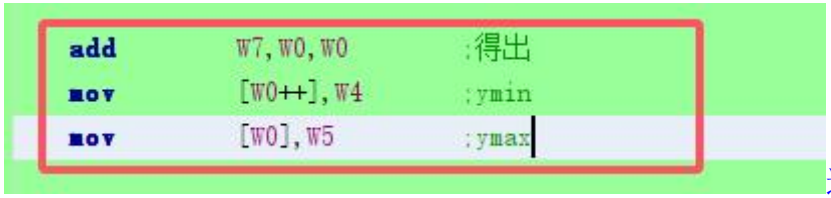
    mul.uu      W6,W5,W6
    repeat      #17
    div.ud      W6,W3   ;w0 -> y - ymin

    add         W4,W0,W0        ;y = y - ymin + ymin

    pop         DSRPAG
```

以上呢，就是对开平方根的函数分析啊。那么，我们调用这个函数的话，就可以求解出矢量的模长了。我们今天就先讲到这里哈，下次课我们来分析 Cordic 数字迭代算法的函数。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问： 
```
add     W7,W0,W0        ;得出
mov     [W0++],W4       ;ymin
mov     [W0],W5         ;ymax
```
这段怎么理解？

答：偶数地址，W0++实际上是+2。W0 中是地址，加上【】就是地址中的数据。

下次课是 10 月 8 日晚上，祝大家节日快乐，大家早点休息，谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。